```
CONSTRAINT valid_vehicle_statuses CHECK (
  VALUE IN ('draft', 'published')
);
```

After running the statement, from psql run \dD to "describe Domains". You've now created a freestanding vehicle_statuses Domain that's not attached to anything.

The Domain extends the Text type, and validates the Draft and Published statuses for a Vehicle.

The DOMAIN object is called vehicle_statuses and it uses a CHECK constraint to constrain the possible values.

To use the vehicle_statuses Domain for the status column, run the following statements. The first statement drops the existing status column so that it can be added back with the domain.

sql/vehicle_status_domain.sql

```
-- drop column so it can be added back
ALTER TABLE vehicles DROP COLUMN status;

-- add column with a Domain called "vehicle_statuses"
ALTER TABLE vehicles
ADD COLUMN status vehicle_statuses
NOT NULL
DEFAULT 'draft';
```

The vehicles.status column now uses a Domain called vehicle_statuses. The column does not allow NULL values and has a default value, and all of that configuration is defined in the database.

Row modifications must pass the CHECK constraint defined in the Domain. The main differences here compared with the Enum from earlier is that the NOT NULL constraint is packaged up along with the Domain. Enums do not have a way to attach additional constraint types to them in this way.

That means this Domain could be used for other tables that have a similar Draft and Published workflow and would benefit from similar Constraint checks. That's pretty cool!

To reset the vehicles table back to the original configuration for Rideshare, run bin/rails db:reset.

You've now implemented a variety of mechanisms to help ensure that data is consistent, correct, and referentially intact. By putting these techniques into practice for your applications when they're new, you can help avoid problems caused by bad data.